CTI: Plataforma Analítica de Planejamento Financeiro

Engenharia de Software e Arquitetura de Sistemas

Design de software e UML aplicados ao produto

Identificação	Entrega 2
Grupo	6 - DevLeaders
Integrantes	Gustavo Pires; Lúcio Vecchio; Luiz Henrique Zaim da Cruz; Eric Aloise
Orientação	Luis Pires
Data de referência	06/11/2026
Código validado	f2bb83637e6f
Repositório	github.com/2026-2-NCC4/Projeto6

1. Arquitetura implementada

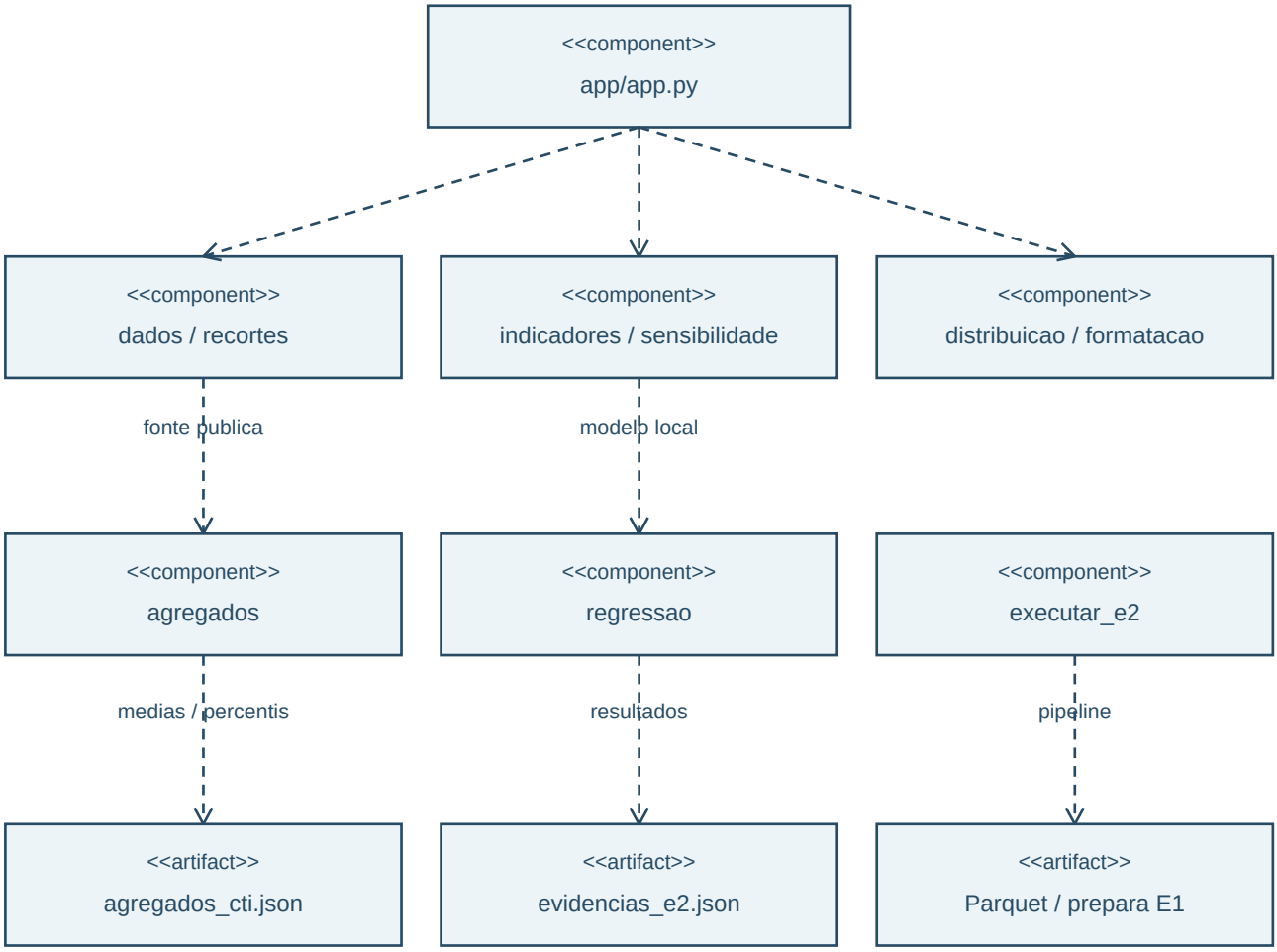
Arquitetura modular em Python: a interface Streamlit coordena serviços de domínio; preparação e validação isolam os dados; cálculos financeiros e regressão são reutilizáveis por CLI, notebook e painel. O sistema não exige banco nem API próprios para o volume atual.

Entradas: CSV CTI autorizado ou Parquet preparado; gerador fictício independente para demonstração; filtros de ano e repetição; hipóteses de desconto, custo, volume e meta. Processamento: validação, recorte, médias anuais, KPIs, simulação e ajuste por trajetórias. Saídas: tabelas, gráficos, indicadores com estado, PDF, evidências e CSV de indicadores agregados reais.

Decisões: reaproveitar prepara.py evita divergência com E1; funções puras separam regra e apresentação; Parquet reduz custo de leitura; cache evita repetir carga e ajuste sem alteração de dados. O modo público não inclui upload ou exportação de dados privados.

Alternativa de banco/API aumentaria manutenção sem requisito atual de concorrência de escrita. A arquitetura admite substituição futura do carregador sem duplicar fórmulas.

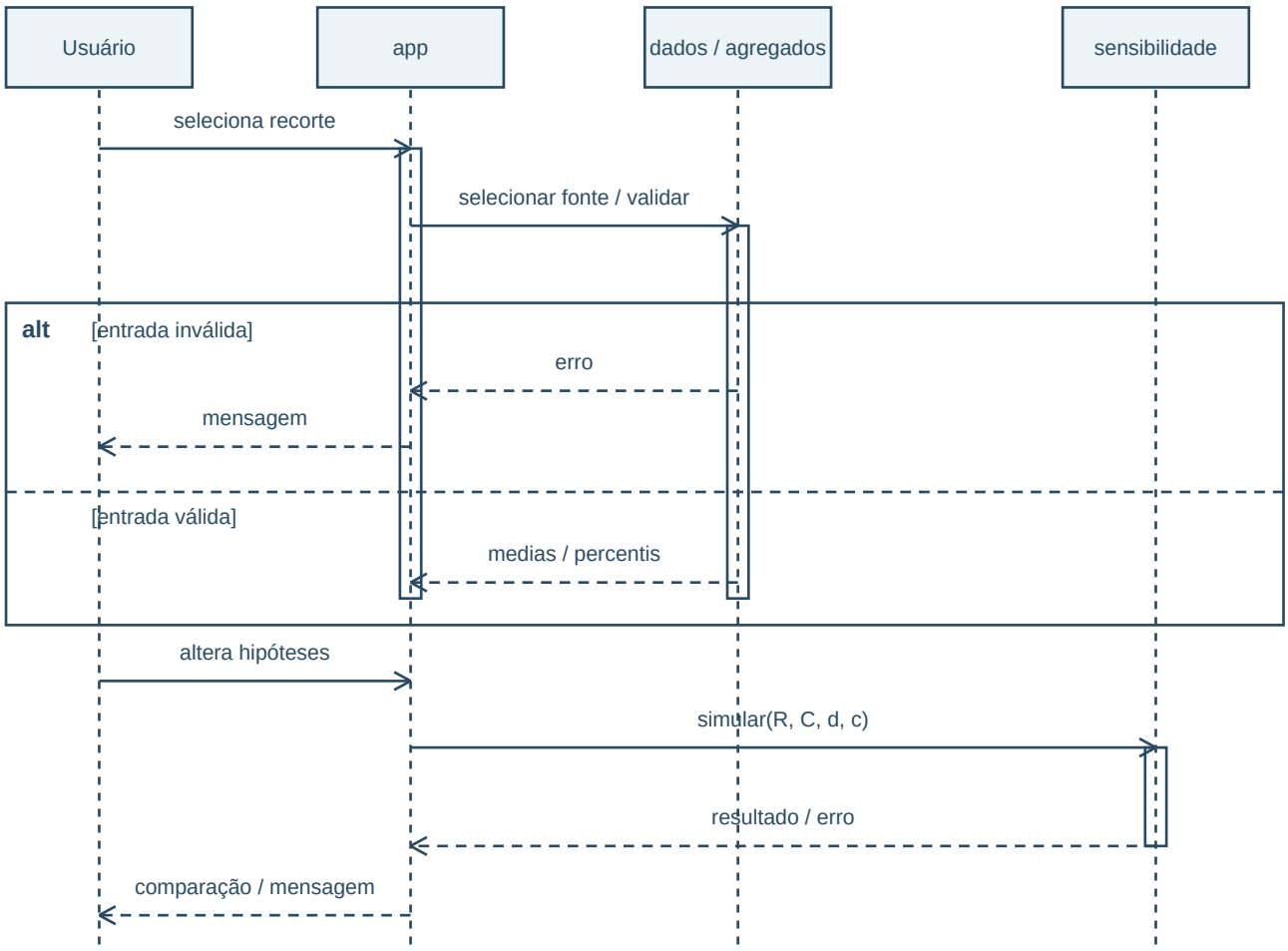
2. UML de componentes



O consumidor aponta para o provedor por dependência tracejada. app utiliza dados/recortes, agregados, indicadores, sensibilidade, distribuição, formatação e regressão. executar_e2 prepara e avalia; a versão pública lê agregados_cti.json e evidencias_e2.json. A base completa é local; demonstração é uma opção explícita.

Fontes editáveis: componentes.puml e sequencia.puml. O desenho representa módulos existentes, não um fluxo genérico.

3. UML de sequência



Participantes: usuário, app, dados/agregados e sensibilidade. Linhas de vida tracejadas, mensagens e retornos; fragmento alt distingue entrada inválida e válida. Resultado com falha é apresentado como mensagem, nunca substituído silenciosamente por zero.

O processamento da simulação recebe cópia dos valores de referência. Restaurar hipóteses muda apenas os controles; não escreve na base. No modo agregado, a regressão lê evidências já calculadas; no modo local, é iniciada por comando explícito. Seu recorte 1-11 é informado separadamente.

4. Rastreabilidade e qualidade

Requisitos FECAP	Implementação / evidência
RF01-RF05: fontes e preparação	prepara.py E1 + dados.py; hash preservado, 15 verificações e cobertura por cenário
RF06-RF07: análise e regressão	E1 descritiva preservada; regressao.py + notebook E2 com 6/6 células
RF08 / RF11: KPIs e cenários	indicadores.py e sensibilidade.py; 216 KPIs + 24 identidades conferidos
RF09-RF10 / RF13: relatório e painel	Quatro PDFs; cinco cartões, evolução, tabelas, filtros e notas
RF12: exportação autorizada	CSV agregado real disponível; base completa e linhas individuais não exportadas
RF14: execução e publicação	Execução local testada; nuvem ainda não verificada
Confiabilidade / manutenção	37 testes aprovados, incluindo entradas inválidas, recortes, fórmulas, modelo, interface e teste negativo real
Reprodução / segurança	Dependências específicas E2; base completa ignorada pelo Git; produção CTI agregada como padrão

A suíte utiliza casos independentes com valores conhecidos e comparação com planilha E1; não verifica apenas que o código executa. Dados essenciais ausentes, chave duplicada, cobertura incompleta e simulação inválida bloqueiam os respectivos resultados.

O acompanhamento operacional e as trocas entre ferramentas são privados, por orientação do grupo. Este PDF registra requisitos implementados e evidências do produto, sem publicar o planejamento interno.

5. Resultados e reflexão

Esperado: integrar as disciplinas em um único painel, preservar a base original, manter consistência de números e preparar publicação. Obtido: aplicação local com cinco KPIs, filtros, financeiro, sensibilidade e regressão; preparação real e comparação financeira sem divergências; notebook executado; documentos gerados.

Divergência relevante: a regressão apresenta colinearidade alta e forte dependência temporal. A solução adotou divisão por trajetória, erros-padrão agrupados e interpretação associativa. Não escondemos essa limitação para obter um indicador de ajuste mais atraente.

Na inspeção visual inicial, cartões com fundo claro sobre tema escuro ficaram ilegíveis. O estilo foi corrigido e os cartões distribuídos em duas linhas, com unidade explícita em milhões. Isso ilustra a diferença entre teste funcional e revisão visual.

As datas e o método de E1 foram mantidos como história, sem inventar reuniões, validação empresarial ou aceites. A nova preparação não sobrescreve relatórios E1. Código específico de plataforma foi separado do ambiente antigo de Windows.

Melhorias futuras: esclarecer convenções CTI, testar estabilidade do modelo em outras partições, avaliar modelos de dependência temporal e acrescentar autenticação somente se houver publicação autorizada de dados restritos. O protótipo não sustenta decisão econômica real sem validação do domínio.

Pendência de implantação: escolha/aprovação da plataforma e teste externo da URL. O funcionamento local não equivale à conclusão de RF14. Banner e documento de extensão permanecem etapa posterior, conforme prioridade do grupo.

Referências: enunciado FINAL FECAP, seções 4-6 e 11; artefatos E1; módulos e testes E2 do repositório. Diagramas e descrições foram confrontados com os arquivos existentes.